

Formal Languages and Compilers

22 July 2024

Using the JFLEX lexer generator and the CUP parser generator, realize a JAVA program capable of recognizing and executing the programming language described in the following.

Input language

The input file is composed of two sections: *header*, and *exec* sections, separated by means of the sequence of characters “####”. Comments are possible, and they are delimited by the starting sequence “<<<” and by the ending sequence “>>>” or C++ style comments (i.e., starting from the sequence “//” to the end of the line).

Header section: lexicon

The *header* section can contain 3 type of tokens, each terminated with the character “;”:

- <tok1>: Starts with “A:”, followed by a hexadecimal number between 0f and 1AB, or by the word “abc”. The first part is followed by 3 to 5 repetitions of the strings “++” or “--” and by a sequence of uppercase alphabetic letters in odd number.
- <tok2>: Starts with “B:”, followed by an odd number of words (at least 3), separated by “&”, “?”, or “!”. Each word is composed of 3 or 6 repetitions of “cd”, “ef”, “gh”, or “ij”.
- <tok3>: Starts with “C:”, followed by a date in the format DD-<month>-YYYY where <month> can be April, May, June, July, August, or September, within the range from 13-April-2024 to 19-September-2024 (remember that the months of April and June have 30 days). Optionally followed by a time in the format :HH:MM between 10:07 and 18:23.

Header section: grammar

In the *header* session each token can appear **in any order and number (also 0)**, with the only exception of <tok2>, which can appear **only 0, 1 or 7 times**. Remember to manage this requirement with grammar.

Athletes section: grammar and semantic

The *exec* section is composed of a list of **at least 4 instructions** in **even** number (i.e., 4, 6,...).

Three kinds of instruction (terminated by “;”) are available:

- <ass>: is composed by an <id> (i.e., a common *C identifier*), a “->”, and a <value> (i.e., a *quoted string between ' characters*). This instruction stores the pair <id>/<value> into a global symbol table. **This symbol table is the only global data structure allowed in all the examination, and it can be written only by an <ass> instruction.** Each time an <ass> instruction is executed <id> and <value> are printed.
- MAX: is the word “MAX” followed by a list of <id> separated with “,”. This instruction prints the <value> with the maximum length between the values associated with the listed <id>. In java, to obtain the length of a string, you can use the .length() method of a String object.

- IF: is the word “IF”, followed by a “[”, a `<bool_exp>`, a “]”, the word “TRUE_STATEMENT”, a “(” a list of `<ass>` and a “)”. It is optionally followed by the word “FALSE_STATEMENT”, a “(” a list of `<ass>` and a “)”. The list of `<ass>` of the TRUE_STATEMENT is executed if `<bool_exp>` is *true*. Instead, the list of `<ass>` of the FALSE_STATEMENT is executed if `<bool_exp>` is *false*. In both cases, for each `<ass>`, use **inherited** attributes to access the result of `<bool_exp>` and decide to execute or not the `<ass>` instruction. `<bool_exp>` is a typical *boolean* expression containing “AND”, “OR”, “NOT” operators, and parenthesis. Operands are “T” (i.e., *true*), “F” (i.e., *false*), or the sequence of tokens `<id>`, “==”, and `<value>`, which returns *true* if the value associated to `<id>` is equal to `<value>`, otherwise it returns *false*.

The translator must produce the output reported in the example. For any detail not specified in the text, follow the example.

Example

Input:

```
// header section
A:1a++++---+---HELLO;           <<< tok1 >>>
B:cdefcd&cdcdcdcdcdcd?ijefij ; <<< tok2 >>>
C:22-July-2024:10:08;           <<< tok3 >>>
A:abc+++++SOS ;                 <<< tok1 >>>
####
// exec section
a -> '1';
b -> '2' ;

IF [ a == '1' OR T AND b == '3' ]
TRUE_STATEMENT (
  c <- 'one';           <<< executed >>>
  d <- 'two';           <<< executed >>>
)
FALSE_STATEMENT (
  c <- 'three' ;       <<< not executed >>>
);

IF [ NOT a=='2' AND NOT NOT a=='1' ]
TRUE_STATEMENT (
  e <- 'four';         <<< executed >>>
);

f <- 'done';

MAX a, f, b; <<< f is the string with a maximum length equal to 6 >>>
```

Output:

```
a '1'
b '2'
c 'one'
d 'two'
e 'four'
f 'done'
'done'
```

Weights: Scanner 8/30; Grammar 9/30; Semantic 10/30